# Capítulo 1: Inicio - Sobre el Control de Versiones

*Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA*

## Introducción

Este capítulo introduce qué es el control de versiones, por qué existe Git, cómo piensa los datos y cómo dejarlo instalado y configurado para empezar a trabajar.

### 1.1 Acerca del Control de Versiones

Un sistema de control de versiones (VCS) registra los cambios hechos sobre un conjunto de archivos a lo largo del tiempo, de modo que se puedan recuperar versiones anteriores, comparar cambios, saber quién modificó qué y cuándo, y volver atrás si algo sale mal.

Existen tres generaciones de sistemas:

- Locales (por ejemplo RCS): guardan las diferencias en una base de datos local. Sirven para una sola persona y no permiten colaborar.
- Centralizados (CVS, Subversion, Perforce): un único servidor contiene todas las versiones y los clientes descargan copias de trabajo. Si el servidor cae o se corrompe, se pierde el historial.
- Distribuidos (Git, Mercurial, Bazaar): cada cliente replica el repositorio completo, con todo su historial. Si un servidor muere, cualquier clon sirve para restaurarlo.

### 1.2 Una breve historia de Git

El núcleo de Linux usó entre 2002 y 2005 el sistema propietario BitKeeper. Cuando la relación comercial terminó, Linus Torvalds y la comunidad crearon su propia herramienta en 2005 con estos objetivos: velocidad, diseño sencillo, soporte robusto para ramificación no lineal (miles de ramas paralelas), totalmente distribuido y capaz de manejar proyectos grandes como el kernel de Linux.

### 1.3 Fundamentos de Git

#### Copias instantáneas, no diferencias

Git no almacena la información como una lista de cambios por archivo, sino como una serie de instantáneas (snapshots) del proyecto completo. En cada confirmación guarda una foto del aspecto de todos los archivos; si un archivo no cambió, solo guarda un enlace al archivo idéntico anterior.

#### Casi todas las operaciones son locales

Como el historial completo está en el disco, consultar el historial, comparar versiones o confirmar cambios no requiere red. Se puede trabajar sin conexión y sincronizar después.

#### Integridad

Todo en Git se verifica con una suma de comprobación SHA-1 (cadena de 40 caracteres hexadecimales) antes de ser almacenado, y se referencia por ese hash. Es imposible cambiar el contenido de un archivo o directorio sin que Git lo detecte.

#### Los tres estados

Un archivo puede estar en tres estados principales:

- Modificado (modified): se cambió pero aún no está marcado para la próxima confirmación.
- Preparado (staged): se marcó la versión actual para que entre en la próxima confirmación.
- Confirmado (committed): los datos están almacenados de forma segura en la base de datos local.

Esto genera tres áreas de trabajo: el directorio de trabajo (working tree), el área de preparación (staging area o índice) y el directorio de Git (.git), donde viven los metadatos y la base de datos de objetos. El flujo básico es: modificar archivos, prepararlos con git add y confirmarlos con git commit.

## 1.4 La Línea de Comandos

Existen muchas interfaces gráficas para Git, pero la línea de comandos es la única que permite ejecutar todos los comandos y es la que sirve de referencia en todo el libro. Quien sabe usar la línea de comandos puede usar cualquier interfaz gráfica, pero no necesariamente al revés.

## 1.5 Instalación de Git

### Linux

```
# Fedora / RHEL
sudo dnf install git-all

# Debian / Ubuntu
sudo apt install git-all
```

### macOS

```
# Con las herramientas de línea de comandos de Xcode
git --version

# O con Homebrew
brew install git
```

### Windows

Se descarga el instalador oficial desde https://git-scm.com/download/win; incluye Git Bash, una terminal con los comandos de Git y utilidades de Unix. Para verificar la instalación:

```
git --version
```

## 1.6 Configurando Git por primera vez

El comando git config permite leer y establecer variables de configuración en tres niveles: --system (todos los usuarios del equipo), --global (el usuario actual) y --local (el repositorio actual; es el nivel por defecto y tiene prioridad sobre los anteriores).

```
git config --global user.name "Ricardo"
git config --global user.email "correo@ejemplo.com"
git config --global core.editor "code --wait"
git config --global init.defaultBranch main

# Revisar la configuración
git config --list
git config user.name
```

> **Nota:** El nombre y el correo quedan registrados en cada commit y no se pueden cambiar después sin reescribir la historia, por eso conviene configurarlos antes de empezar.

### 1.7 ¿Cómo obtener ayuda?

```
git help <comando>
git <comando> --help
man git-<comando>

# Ayuda corta, solo opciones disponibles
git add -h
```

## Resumen

Git es un sistema de control de versiones distribuido que guarda instantáneas del proyecto, trabaja de forma local, garantiza la integridad con SHA-1 y organiza el trabajo en tres estados (modificado, preparado, confirmado). Con Git instalado y configurado con user.name y user.email ya se puede comenzar a crear repositorios.

### Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 1.